Directive-Based GPU Programming with OpenMP and OpenACC

The Pawsey Supercomputing Centre is an unincorporated joint venture between

CSIRO · Curtin University · ECU Edith Cowan University · Murdoch University · The University of Western Australia

and proudly funded by

NCRIS National Research Infrastructure for Australia, An Australian Government Initiative · Government of Western Australia

Pawsey

Agenda

- **Introduction to Directive-Based GPU programming**
- Development Cycle
- OpenMP vs OpenACC
- OpenMP Directives
- OpenACC Directives
- Hands-on Session

Ways to accelerate code on GPUs

Applications

3rd Party Libraries

Directives

Programming Languages

Ways to Accelerate code on GPUs

Applications

3rd Party Libraries

- Most performance
- Easy to use (Drop in Acceleration)
- Less flexible

Directives

- Easy to use
- Portable GPU/non-GPU devices
- This model was for many years considered as not the most performant

Programming Languages

- Most performance
- More flexibility
- Can use vendor libraries
- Not very portable
- More effort

Directive-Based GPU Programming
Main Objectives

- **Performance**
- **Portability**

OpenMP attempt to introduce a good balance between the two in your code.

Directive-Based GPU Programming
Parallelism

Directive-based programming models allow programmer to express parallelism in code allowing:

- **Simple compiler hints by the programmer**
- **Compiler to generate threaded code based on these hints**
- **Ignorant complier to treat directives as comments**

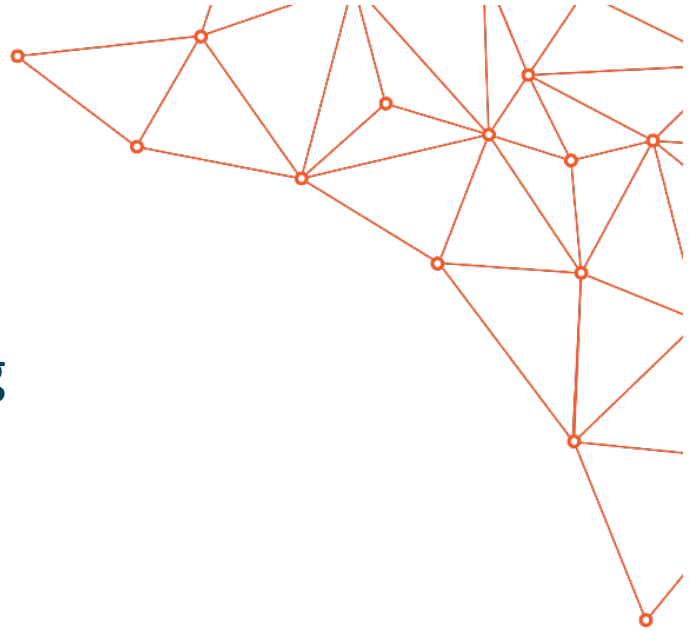
Directive-Based GPU Programming
Benefits

- Incremental parallelism
- Single source for serial and parallel program
- Easier to learn then the low level APIs and Languages (e.g. HIP, OpenCL)
- Portable to more then one kind of architectures/accelerators

Example

```
#pragma omp target map(to: x[0:ROWS][0:COLS]) map(tofrom: y[0:ROWS][0:COLS])
#pragma omp parallel for collapse(2)
for (int i = 0; i < ROWS; i++) {
    for (int j = 0; j < COLS; j++) {
        y[i][j] += x[i][j];
    }
}
```

Agenda

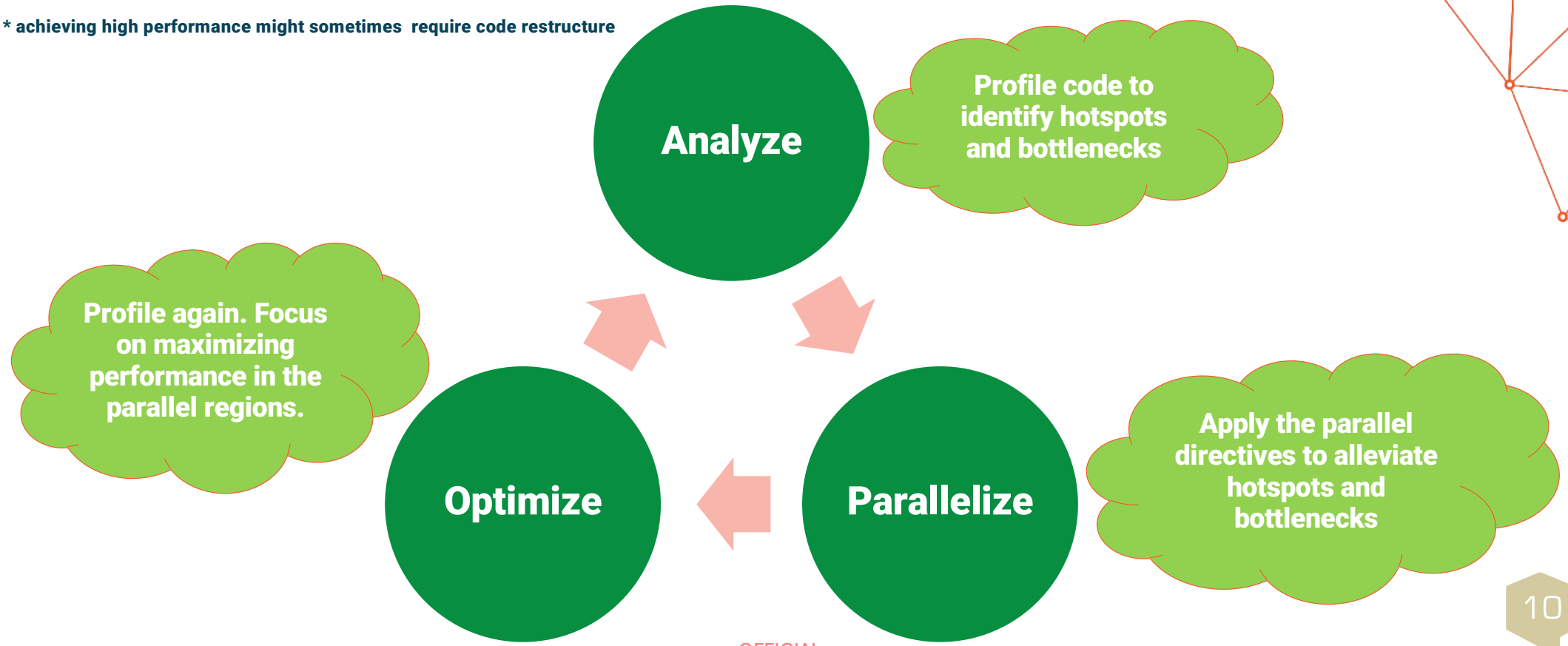


- Introduction to Directive-Based GPU programming
- **Development Cycle**
- OpenMP vs OpenACC
- OpenMP Directives
- Hands-on Session

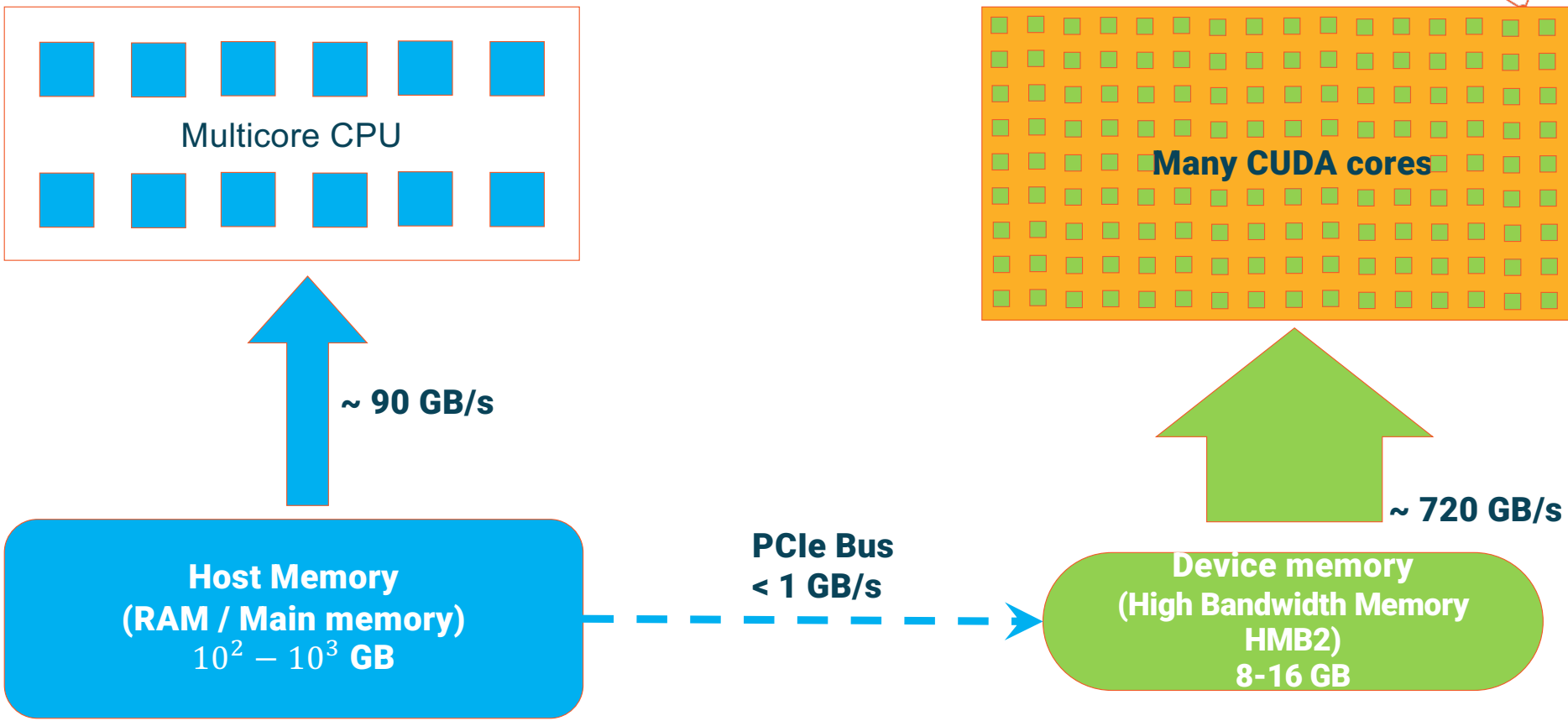
Development Cycle

Parallelism is added incrementally: i.e. the sequential program evolves into a parallel program *

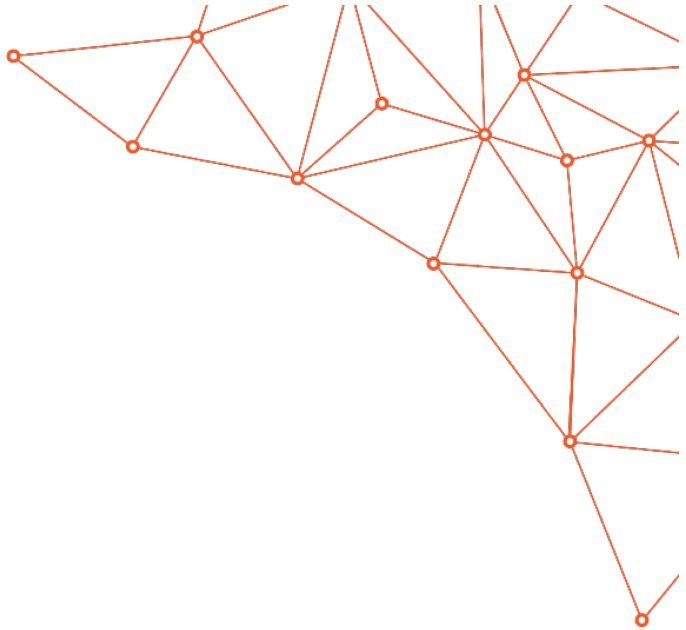
*** achieving high performance might sometimes require code restructure**



Architectural bottleneck to watch for



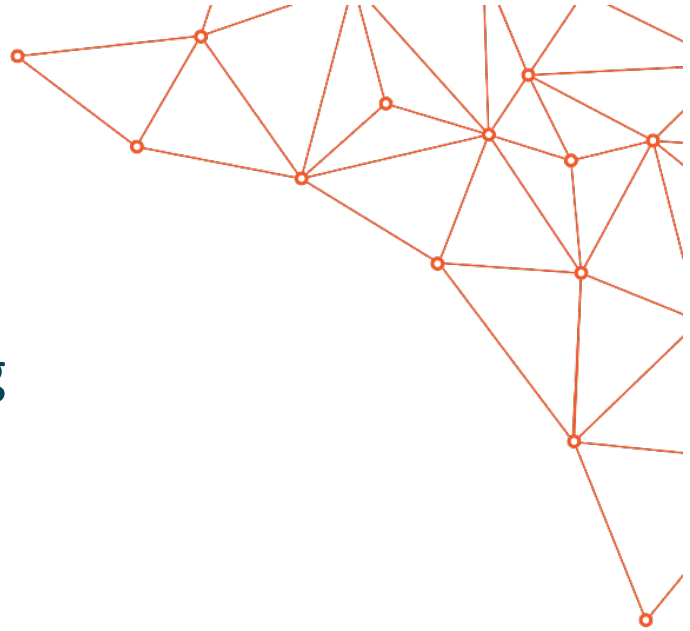
Optimize Data locality



- Minimize memory copy from Host to Device
- Avoid duplication if possible
  - Only copy from or to Host what is valuable
  - Maximize reuse of the copied data
- Copy only what's needed
- Contiguity in memory is your friend
  - Compilers like it, increases cache reuse
- Introduce more FLOPs per byte if possible

Note: Compiler will do it's best to optimize the code based on directives provided by the programmer; in most cases compiler will introduce some implicit data movement between host and device

Agenda



- Introduction to Directive-Based GPU programming
- Development Cycle
- **OpenMP vs OpenACC**
- OpenMP Directives
- OpenACC Directives
- Hands-on Session

OpenMP

Directive-Based GPU Programming Models

OpenACC

GPU Support in OpenMP

- The CPU version of OpenMP was initially developed in 1997 to simplify the writing of multithreaded programs in Fortran, C and C++
- Consists of a collection of compiler directives, library routines and environment variables
- OpenMP introduced support for offloading to GPU devices in 4.5 Standard (November 2015)
- Support for OpenMP 4.5 device offloading in various compilers:
  - IBM XL compiler
  - Clang
  - GCC
  - PGI available soon (?)

https://www.openmp.org/resources/openmp-compilers-tools/

GPU Support in OpenMP: C/C++

- On Setonix for c/c++ applications there are two ways to get OpenMP GPU offload working:
  - Via clang provided by PrgEnv-cray/8.6.0 (That is Cray clang version 19.0.0)

```
module load PrgEnv-cray/8.3.3 rocm/5.7.1 craype-accel-amd-gfx90a
cc –O3 -fopenmp --offload-arch=gfx90a   veccopy.c -o veccopy
```

  - Via clang which is included in llvm which comes with rocm (That is AMD clang version 18.0.0)

```
hipcc -O3 -fopenmp --offload-arch=gfx90a  veccopy.c -o veccopy
```

    hipcc is a wrapper for /opt/rocm/llvm/bin/clang

https://www.openmp.org/resources/openmp-compilers-tools/

GPU Support in OpenMP: Fortran

- On Setonix for Fortran applications there are currently two ways to get OpenMP GPU offload working:
  - Via ftn provided by PrgEnv-cray/8.6.0 (That is Cray Fortran version 19.0.0)

```
module load PrgEnv-cray/8.6.0 rocm/6.3.0 craype-accel-amd-gfx90a
ftn -h omp simple_offload.f95 -o simple_offload
```

  - Via flang which is included in llvm which comes with rocm (That is AMD flang version 18.0.0)

```
/opt/rocm/llvm/bin/flang -fopenmp --offload-arch=gfx90a simple_offload.f95 -o simple_offload
```

https://www.openmp.org/resources/openmp-compilers-tools/

GPU Support in OpenACC

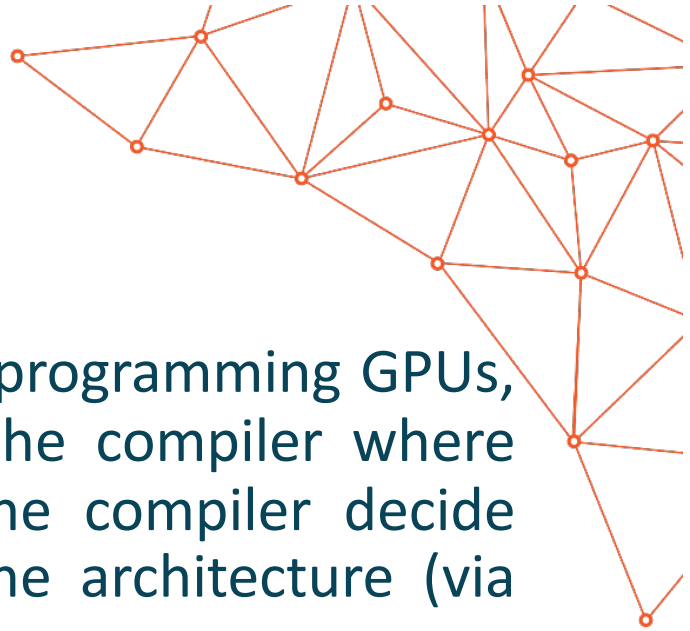
- OpenACC is a programming standard that describes a collection of compiler directives to organizes loops and regions of codes to be offloaded from a host CPU to an attached accelerator.
- Developed by the Portland Group (PGI), Cray, CAPS and NVIDIA
- OpenACC 3.4 Standard is current and available online
  - http://www.openacc.org
  - PGI is usually supporting the most recent standard features
  - Cray compilers also provide support for OpenACC (2.0)
  - GCC and Clang compilers support OpenACC on selected accelerators
  - On Setonix only Cray Fortran supports openacc

- **Note**: OpenACC can be also used to parallelise applications on CPU cores via threads

OpenACC vs OpenMP

- Most of the directives introduced in OpenACC have their equivalents in OpenMP
- Important exception: kernels directive(to be discussed later)

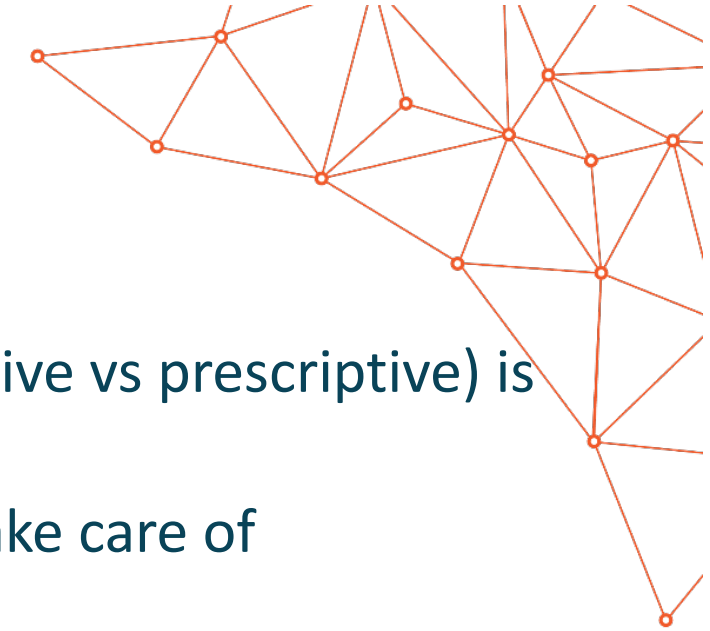
- Concepts are very similar
- Performance and compiler support for various platforms can be different



OpenACC vs OpenMP

- OpenACC is said to be a descriptive approach to programming GPUs, where the programmer uses directives to tell the compiler where data-independent loops are located and lets the compiler decide how/where to parallelize the loops based on the architecture (via compiler flags).

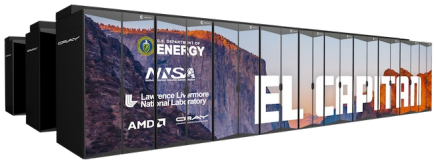
- OpenMP, on the other hand, is said to be a prescriptive approach to GPU programming, where the programmer uses directives to more explicitly tell the compiler how/where to parallelize the loops, instead of letting the compiler decide.

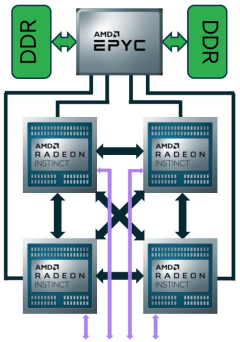


OpenACC vs OpenMP

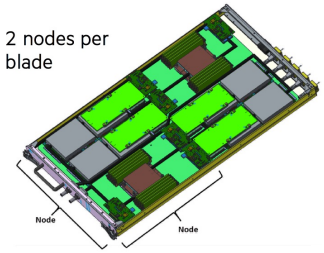
- It is really hard to judge which approach (descriptive vs prescriptive) is better.
- On the one hand we would like the compiler to take care of optimisations as much as possible.
- On the other hand programmers **must** have a clear understanding on what transformations were made to their code.
- **We claim that creating a highly optimised GPU code requires a very similar effort in both OpenACC and OpenMP approaches**.

Next-Generation Supercomputers





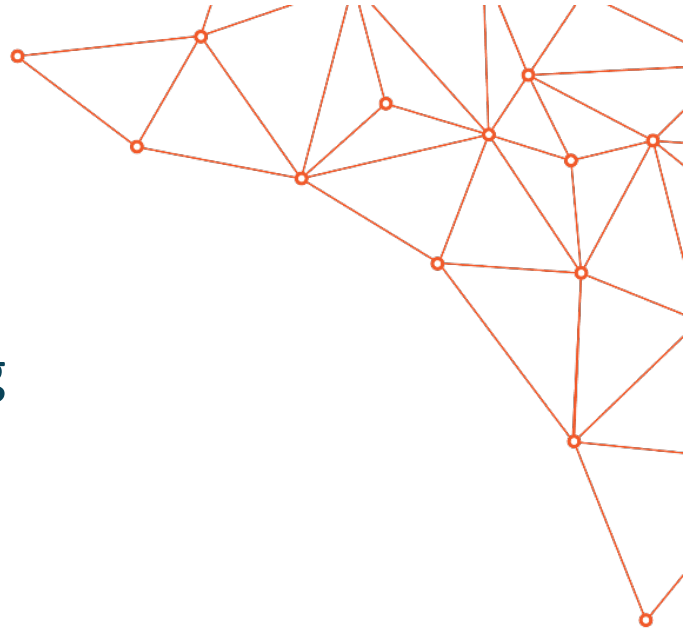
AMD GPU (ORNL)



Migration Path from Summit to Frontier

Summit	Frontier	Comments
CUDA C/C++	HIP C/C++	HIP provides tools to help port existing CUDA codes to the HIP layer. HIP is not intended to be a drop-in replacement for CUDA, and developers should expect to do some manual coding and performance tuning work to complete the port.
OpenACC	OpenMP (offload)	OpenACC codes can be migrated to OpenMP (offload) for Frontier. *Direct support for OpenACC on Frontier is still under discussion.*
OpenMP (offload)	OpenMP (offload)	Virtually the same on Summit and Frontier
FORTRAN w/CUDA C/C++	FORTRAN w/HIP C/C++	As with CUDA, this will require interfaces to the C/C++ API calls
CUDA FORTRAN	FORTRAN w/HIP C/C++	As with CUDA, this will require interfaces to the C/C++ API calls

Agenda



- Introduction to Directive-Based GPU programming
- Development Cycle
- OpenMP vs OpenACC
- **OpenMP directives**
- OpenMP directives
- Hands-on Session

`#pragma omp target teams`

- Initiate parallel execution
- In the case of GPU accelerators:
  - The compiler will generate 1 or more parallel teams (mapped to SMs)
  - The teams will execute redundantly (on master thread of each SM)

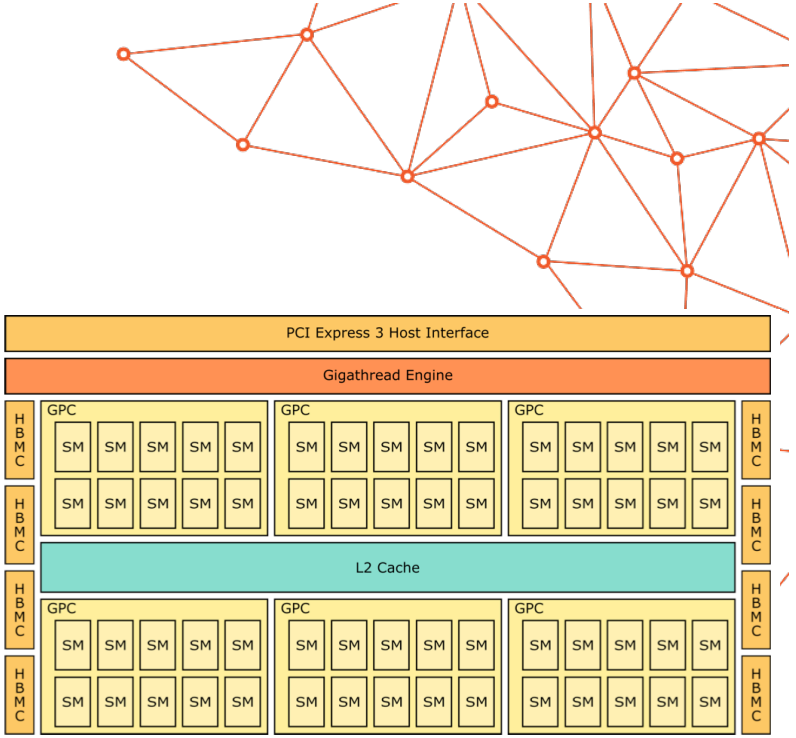


Figure: A schematic representation of the GPU architecture.

`#pragma omp distribute`

- Initiate parallel sharing of work
- Higher level parallelism across teams
- Shares the iterations of the loop or loops across the teams of the parallel region
- In the case of GPU accelerators:
  - Shares the iterations of the loop or loops across the SMs

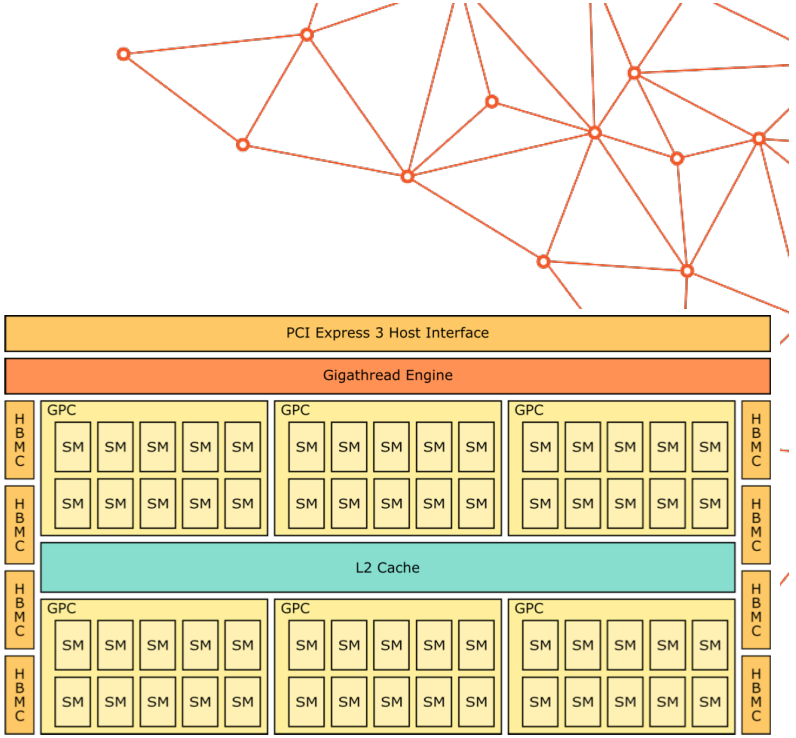


Figure: A schematic representation of the GPU architecture.

`#pragma omp parallel for`

- Initiate parallel sharing of work across threads of the team
- Lower level parallelism
- Shares the iterations of the loop or loops across the threads in team
- In the case of GPU accelerators:
  - Shares the iterations of the loop or loops across the threads of SMs

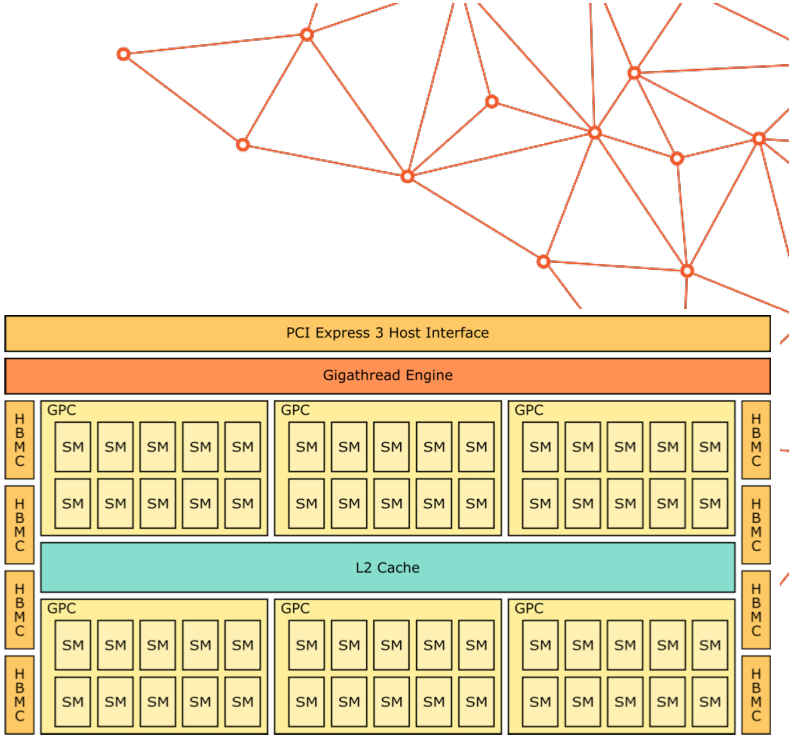


Figure: A schematic representation of the GPU architecture.

`#pragma omp simd`

- Initiate parallel sharing of work across threads of the team in vector SIMD mode
- Lower level parallelism
- Shares the iterations of the loop or loops across the threads in team

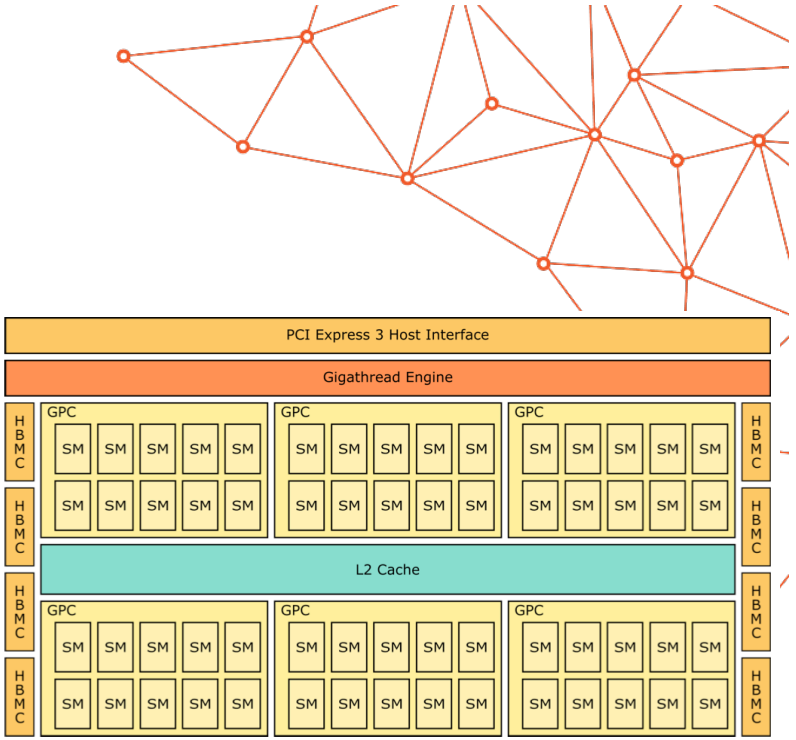


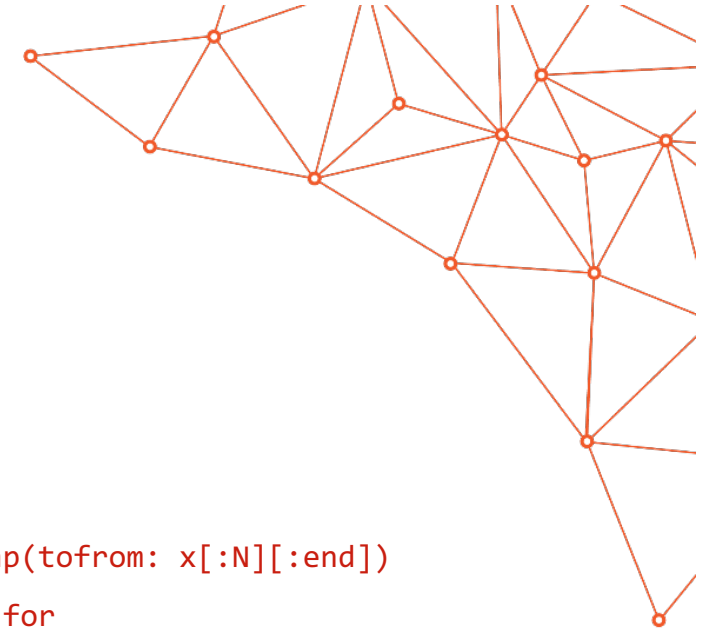
Figure: A schematic representation of the GPU architecture.

OFFICIAL

Data transfers

```
#pragma omp data map(to:x[0:ROWS][0:COLS], y[0:ROWS][0:COLS]))\
map(from:y[0:ROWS][0:COLS])
{
#pragma omp target teams distribute parallel for
                for (i=0; i<ROWS; i++)
                        for (j=0; j<COLS; j++)
                                y[i][j] += x[i][j];
}
```

OpenMP



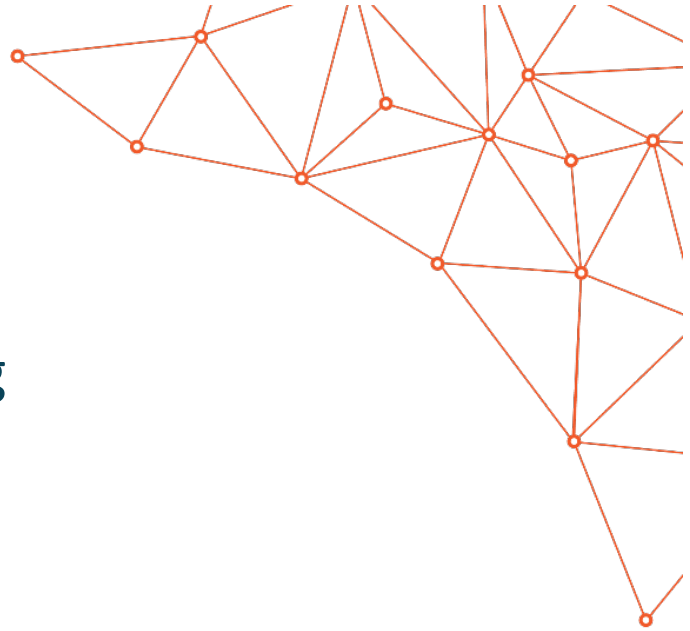
#pragma omp target update

- Some data synchronization is required between H and D while computation in progress.
  - E.g. Check-pointing or writing results on disk.

- update directive copies data allocated on device memory to the CPU memory or vice versa.
  - It may appear within a data region, implicit or explicit.

```
#pragma omp target map(tofrom: x[:N][:end])
#pragma omp parallel for
for(int i = 0; i < N; i++) {
          // compute all values of x[i][0:end]
}
```

Agenda



- Introduction to Directive-Based GPU programming
- Development Cycle
- OpenMP vs OpenACC
- OpenMP Directives
- **OpenACC Directives**
- Hands-on Session

`#pragma acc parallel`

- Initiate parallel execution on accelerator
- In the case of GPU accelerators:
  - The compiler creates one or more gangs of parallel work
  - Gangs map conceptually to SMs/Cus
  - Workers/vector lanes execute within each gang

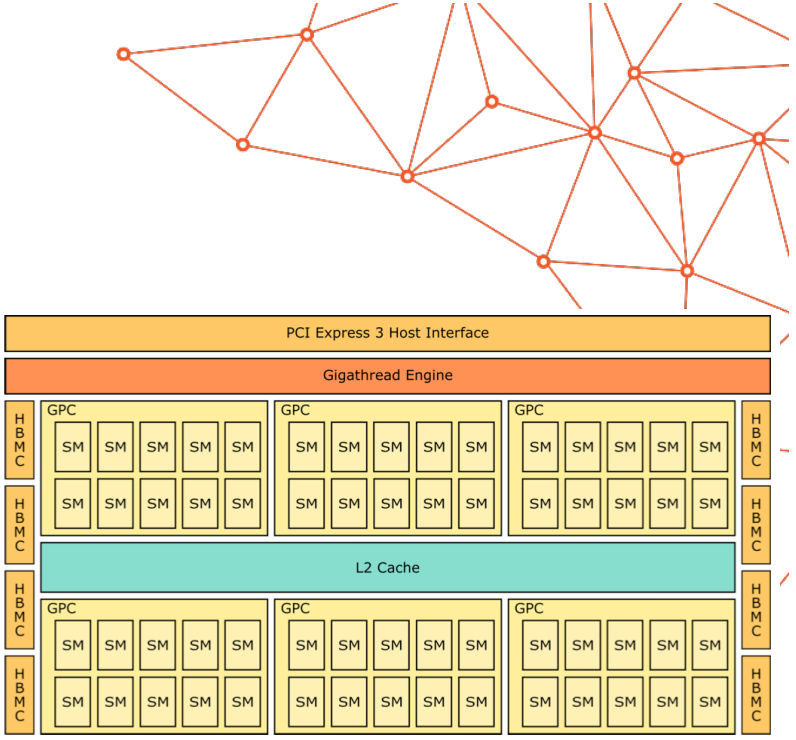


Figure: A schematic representation of the GPU architecture.

`#pragma acc loop gang`

- Initiate parallel sharing of work
- Higher level parallelism across gangs
- Shares the iterations of the loop or loops across the gangs in the parallel region
- In the case of GPU accelerators:
  - Distributes loop iterations across SMs/CUs

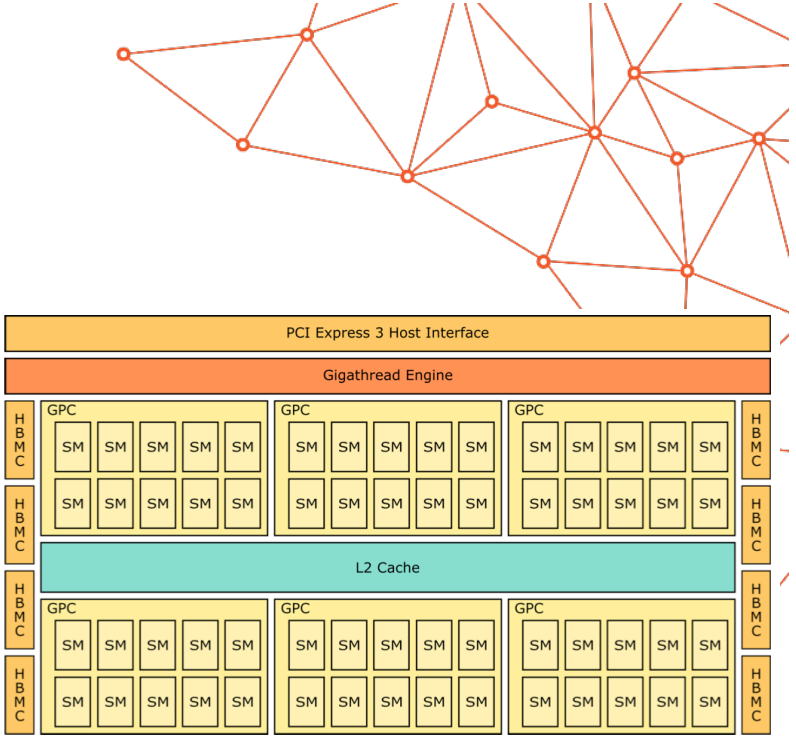


Figure: A schematic representation of the GPU architecture.

`#pragma acc loop worker`

- Initiate parallel sharing of work across workers of the gang
- Lower level parallelism
- Shares the iterations of the loop or loops across the workers in gang
- In the case of GPU accelerators:
  - Maps conceptually to threads/warps within SMs/CUs

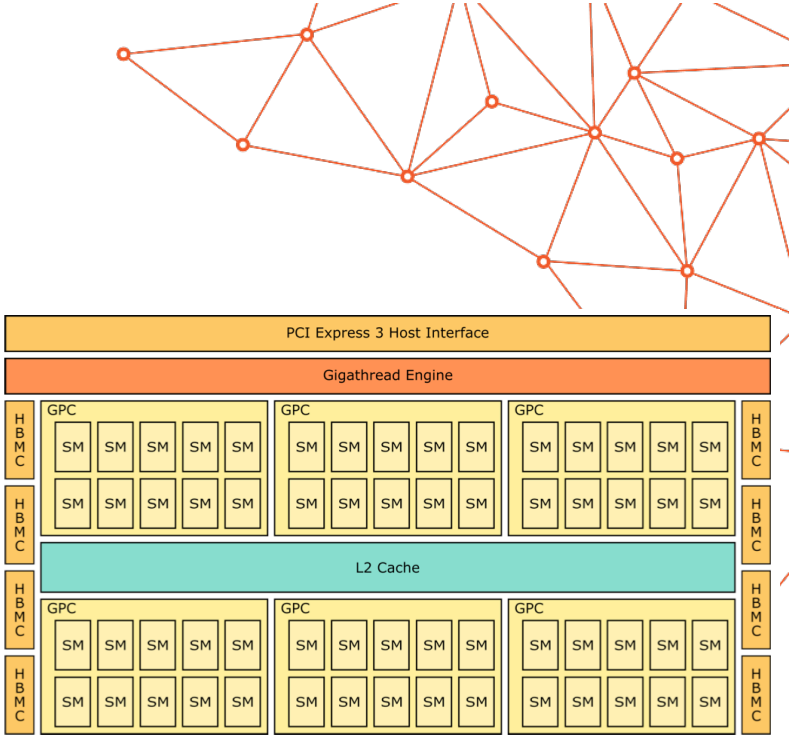


Figure: A schematic representation of the GPU architecture.

#pragma acc loop vector

- Initiate vector-level sharing of work in SIMD mode
- Lowest level parallelism
- Shares the iterations of the loop or loops across vector lanes in a gang/worker

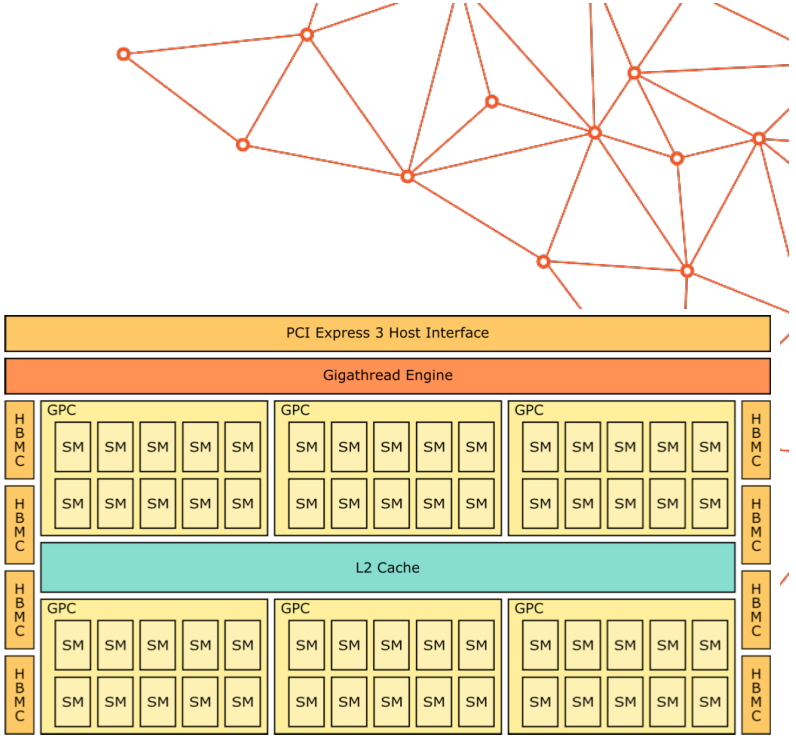


Figure: A schematic representation of the GPU architecture.

- In the case of GPU accelerators:
  - Maps conceptually to lanes/threads of a warp/wavefront

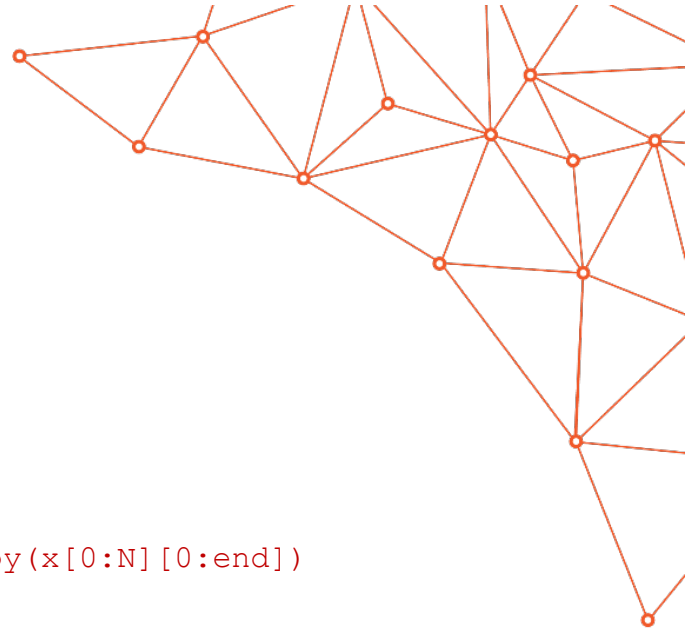
```
#pragma omp target teams  -> #pragma acc parallel
#pragma omp distribute    -> #pragma acc loop gang
#pragma omp parallel for  -> #pragma acc loop worker
#pragma omp simd          -> #pragma acc loop vector
```

Data transfers

```
#pragma acc data copyin(x[0:ROWS][0:COLS]) \
                  copy(y[0:ROWS][0:COLS])

{
#pragma acc parallel loop collapse(2)
               for (i=0; i<ROWS; i++)
                       for (j=0; j<COLS; j++)
                               y[i][j] += x[i][j];
}
```

OpenMP



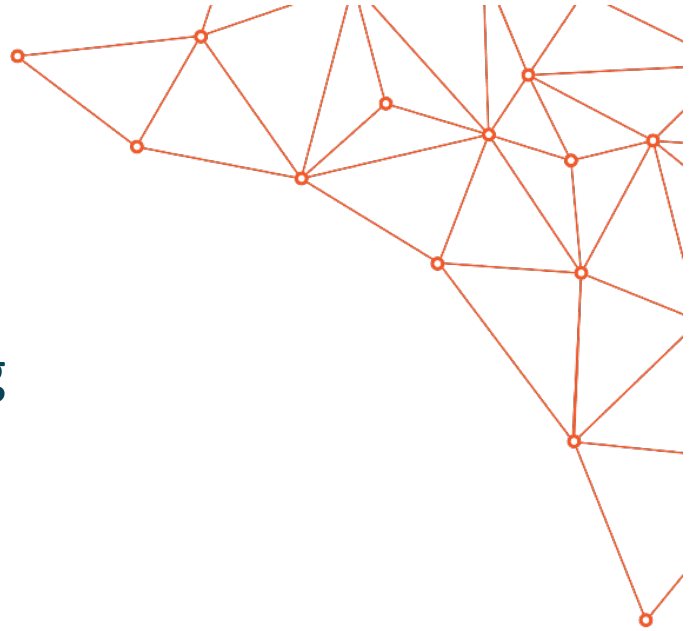
#pragma acc update

- Some data synchronization is required between Host and Device while computation in progress.
  - E.g. Check-pointing or writing results on disk.

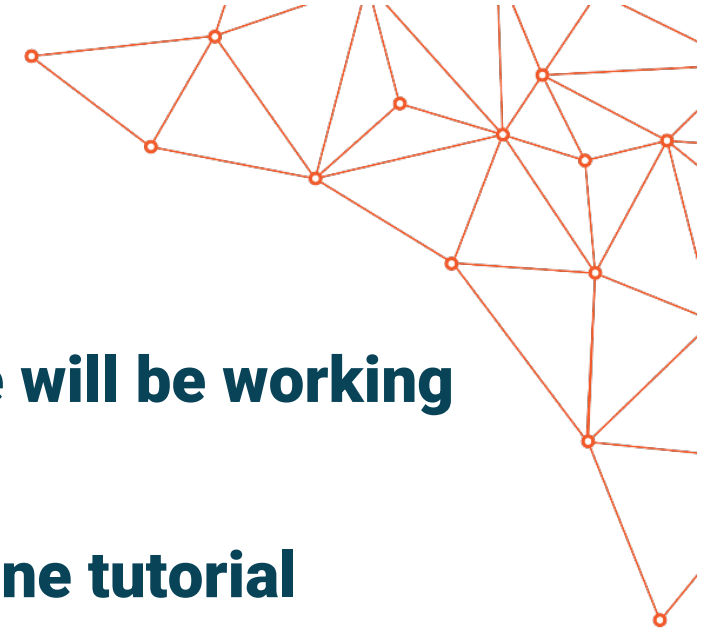
- The update directive copies data allocated on device memory to the CPU memory or vice versa.
  - self(...) : device -> host
  - device(...): host -> device

```
#pragma acc data copy(x[0:N][0:end])
{
#pragma acc parallel loop
for(int i = 0; i < N; i++) {
        // compute all values of x[i][0:end]
}
#pragma acc update self(x[0:N][0:end])
}
```

Agenda



- Introduction to Directive-Based GPU programming
- Development Cycle
- OpenMP vs OpenACC
- OpenMP Directives
- OpenACC Directives
- **Hands-on Session**



Hands-on Session

During the hands-on session of the lecture we will be working with the

“OpenMP and OpenACC GPU offloading” on-line tutorial

https://github.com/PawseySC/OpenMP-and-OpenACC-offloading

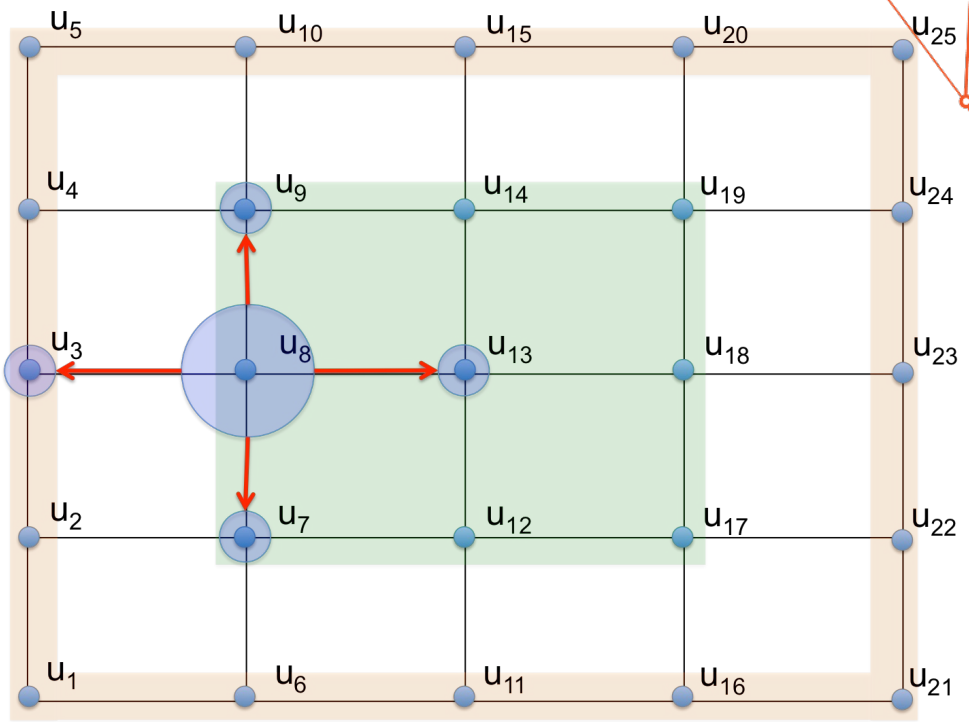
Example problem: Laplace Equation solver

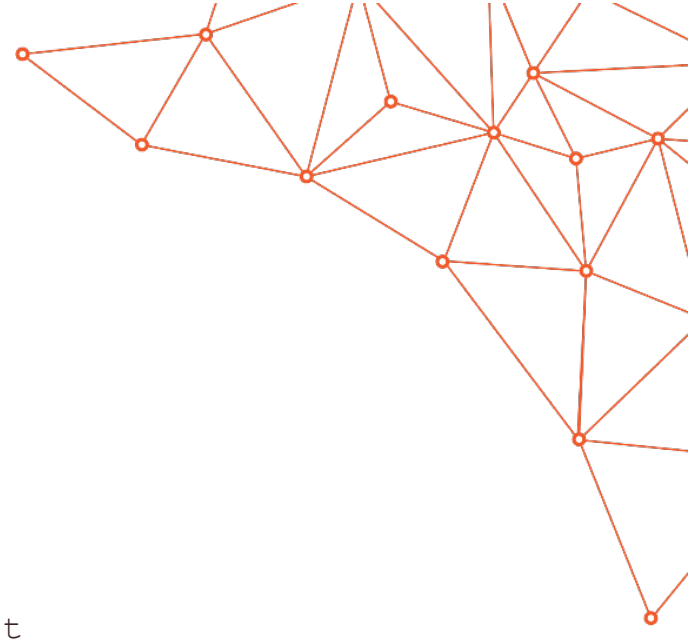
- **Heating a metal plate – which can be simulated by solving Laplace equation**

$$\nabla^2 u(x, y) = 0$$

- **We are interested in the steady state response**

- **Can be solved on a stencil or a 2D grid with iterative process:**
  - **in each iteration temperature in every i-th grid point is an average of its 4 neighbours (up, down, left and right)**





Downloading exercises

- **Login to Setonix via ssh:**

```
ssh username@setonix.pawsey.org.au
```

- **Change directory to $MYSCRATCH**

```
cd $MYSCRATCH
```

- **Use git to download the exercise material:**

```
git clone https://github.com/PawseySC/OpenMP-and-OpenACC-offloading.git
```

- **Change into the downloaded directory:**

```
cd OpenMP-and-OpenACC-offloading
```

- **List the contents of the directory:**

```
ls
```

This tutorial provides a step-by-step guide to programming GPUs with the use of offloading mechanism available in OpenMP.

Step-by-step guide: https://pawseysc.github.io/OpenMP-and-OpenACC-offloading

How is it organised?

https://github.com/PawseySC/OpenMP-and-OpenACC-offloading

- Each episode starts with a separate hands-on directory
  - Each episode starts with the implementation of the previous episode
  - For good learning experience **please do not look into future episodes solutions**, but follow the flow of the tutorial
- For most of the episodes there is an OpenMP directory and codes written in c and Fortran
  - You can decide to choose c or Fortran or both – there will be plenty of time for this
- Each episode contains Makefiles and run.slurm script
- The run.slurm script will do all the magic (it will compile, run the code and profiling if necessary)
  - So, all you need to concentrate on is code development
- Remember to use the reservation when submitting the script:
  - sbatch run.slurm